

Pemrograman Berorientasi Objek

Dany Candra Febrianto, S.Kom., M.Eng.

Minggu 02 : Encapsulation Concept

*Hidup hanya bisa dipahami dengan menoleh ke belakang,
tapi harus dijalani dengan menatap ke depan*

-Søren Kierkegaard, (Journals, 1843)-

Rewritten by Tim Satgas Karakter FIF

*life can only be understood backwards, but it must be lived
forwards*

-Søren Kierkegaard, (Journals, 1843)-

Rewritten by Tim Satgas Karakter FIF

Review Pertemuan Sebelumnya

- Kelas & Objek
 - Attribute/Properties
 - Behaviour/Procedur/Function/Method
- Prinsip Desain PBO
 - Encapsulation
 - Abstraction
 - Inheritance
 - Polymorphsm

Outline (Encapsulation)

1. Access Modifier / Visibility
2. Constructor
3. Encapsulation Concept
4. Modularity & Package

Access Modifier / Visibility

- Access Modifier / Visibility merupakan kemampuan untuk mengontrol sebuah akses ke **kelas**, **atribut**, **prosedur** dan **fungsi**.
- Java mendukung beberapa visibility, yaitu :
 - Default
 - Public
 - Private, dan
 - Protected

private

- Hanya bisa diakses dalam class yang sama.
- Biasanya dipakai untuk atribut.



```
class Mahasiswa {  
    private String nama;  
}
```

Default (tanpa keyword)

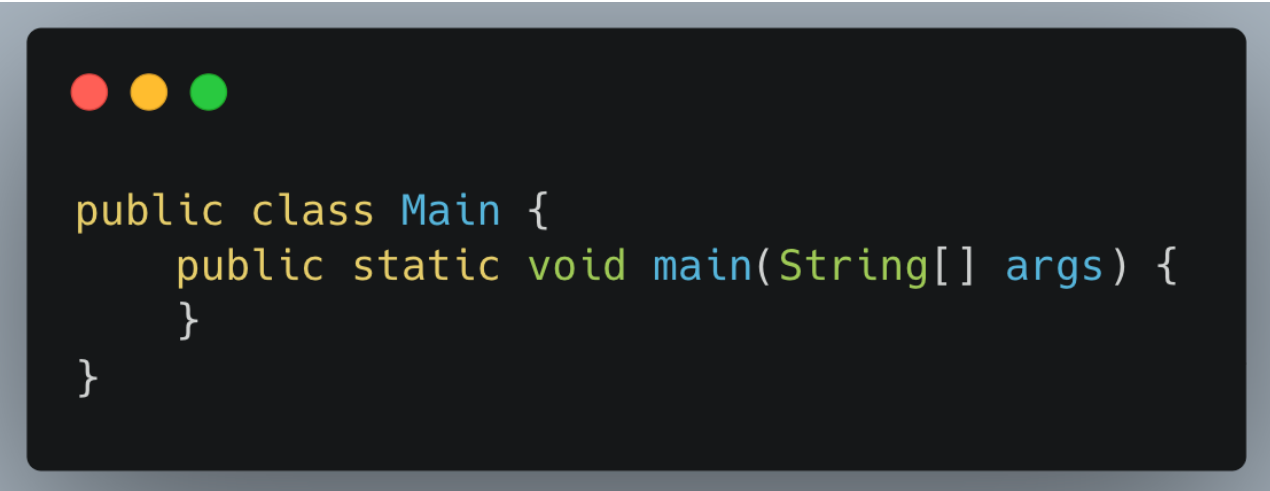
- Bisa diakses oleh class lain dalam package yang sama.
- Tidak bisa diakses dari package berbeda.



```
class Mahasiswa {  
    String jurusan;  
}
```


public

- Bisa diakses dari mana saja.
- Biasanya dipakai untuk class utama dan method umum.



```
public class Main {  
    public static void main(String[] args) {  
    }  
}
```

protected

- Bisa diakses dalam package yang sama.
- Bisa diakses oleh subclass walaupun beda package.
- Biasanya dipakai saat konsep inheritance.

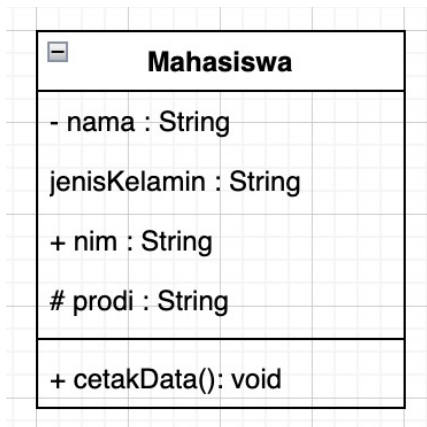


```
public class Mahasiswa {  
    protected String fakultas;  
}
```

Hak Akses

Visibility	Kelas	Paket	Sub Kelas	Umum
Private	Y			
Default	Y	Y		
Protected	Y	Y	Y	
Public	Y	Y	Y	Y

Penerapan Access Modifier



Class Diagram



```
public class Mahasiswa {

    private String nama;

    String jenisKelamin;

    public String nim;

    protected String prodi;

    public void cetakData() {
        System.out.println(nama);
        System.out.println(nim);
        System.out.println(prodi);
        System.out.println(jenisKelamin);
    }

}
```

Implementasi
Code Java

Encapsulation

Encapsulation adalah konsep dalam OOP yang menyembunyikan *data suatu objek* agar tidak bisa diakses langsung dari luar, kecuali melalui method yang telah ditentukan. (**getter & setter**)

Tujuan *Encapsulation*

- Memastikan atribut objek tidak dapat diubah dan diakses secara langsung.
- Hanya prosedur dan fungsi tertentu yang diberi izin untuk mengakses dan mengubah atribut objek.

Contoh *Encapsulation*

```
public class Mahasiswa {  
    private String nim;  
    private String nama;  
  
    public Mahasiswa() {  
    }  
  
    public String getNim() {  
        return nim;  
    }  
  
    public void setNim(String nim) {  
        this.nim = nim;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public void setNama(String nama)  
{  
        this.nama = nama;  
    }  
}
```

Constructor

Dalam **Object-Oriented Programming (OOP)**, **constructor** adalah metode/prosedur khusus yang digunakan untuk menginisialisasi objek saat pertama kali dibuat.

Constructor memiliki **nama yang sama dengan nama kelas** dan **tidak memiliki tipe pengembalian (return type)**, bahkan **void pun tidak**.

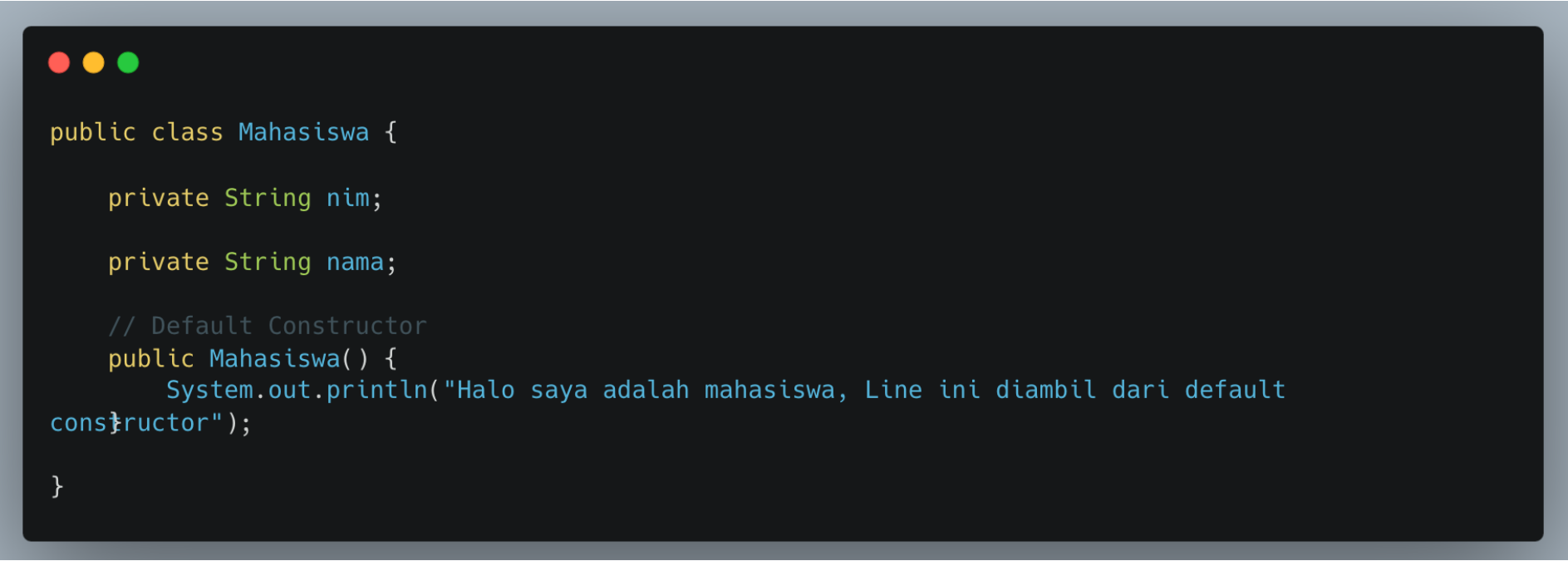
Ciri-Ciri Constructor

- 1. Nama constructor harus sama dengan nama kelas.**
- 2. Tidak memiliki return type (termasuk void).**
- 3. Otomatis dipanggil saat objek dibuat dengan keyword new.**
- 4. Bisa memiliki parameter untuk mengatur nilai awal objek.**
- 5. Bisa dibuat lebih dari satu (Constructor Overloading).**
- 6. Jika tidak ada constructor yang dibuat, Java akan menyediakan default constructor secara otomatis.**

Default Constructor

- Jika sebuah kelas tidak mendefinisikan konstruktor, kompiler akan menyediakan konstruktor default tanpa argumen.
- Jika sebuah kelas diberikan konstruktor baru, maka konstruktor default tidak dapat digunakan.

Contoh Default Constructor



```
public class Mahasiswa {  
  
    private String nim;  
  
    private String nama;  
  
    // Default Constructor  
    public Mahasiswa() {  
        System.out.println("Halo saya adalah mahasiswa, Line ini diambil dari default  
constructor");  
    }  
}
```

Constructor Berparameter

- Constructor yang memiliki parameter, dan parameter tersebut akan diisi ketika objectnya dari kelas tersebut dibuat.

Contoh Constructor Berparameter

```
public class Mahasiswa {  
  
    private String nim;  
  
    private String nama;  
  
    // Parameter Constructor  
    public Mahasiswa(String nim, String nama) {  
        this.nim = nim;  
        this.nama = nama;  
        System.out.println("Halo saya adalah  
                            "+nama+" dengan nim "+nim+" ,  
                            Line ini diambil dari paramater  
constructor");  
    }  
}
```

Overloading Constructor

- Di Java, metode dapat dioverload. **Method overloading** adalah dua atau lebih metode yang memiliki nama yang sama tetapi memiliki parameter yang berbeda.
- Kita dapat membuat lebih dari satu konstruktor dengan melakukan **overloading** terhadapnya menggunakan konstruktor lain dengan parameter yang berbeda.
- Dalam banyak situasi, kita ingin membuat objek dari suatu kelas dengan berbagai kumpulan data awal yang berbeda.

Contoh Overloading Constructor

```
public class Mahasiswa {  
  
    private String nim;  
  
    private String nama;  
  
    // Default Constructor  
    public Mahasiswa() {  
        System.out.println("Halo saya adalah mahasiswa, Line ini diambil dari default constructor");  
    }  
  
    // Parameter Constructor  
    public Mahasiswa(String nim, String nama) {  
        this.nim = nim;  
        this.nama = nama;  
        System.out.println("Halo saya adalah "+nama+" dengan nim "+nim+" , Line ini diambil dari  
parameter constructor");  
    }  
}
```

Modularity



Modularity

- **Modularity** dalam **Object-Oriented Programming (OOP)** adalah prinsip pemisahan program menjadi bagian-bagian kecil yang independen (**modul**) yang dapat dikembangkan, diuji, dan digunakan kembali secara terpisah.
- Setiap **modul** biasanya berupa **kelas atau objek** yang memiliki tugas spesifik dan dapat berinteraksi dengan modul lainnya melalui **interface atau metode publik**.

Package

- Di Java, **package** digunakan untuk mengelompokkan kelas-kelas yang memiliki keterkaitan fungsional ke dalam satu modul. Hal ini memungkinkan:
 - ✓ **Organisasi kode lebih baik** → Kelas dengan fungsi terkait dikelompokkan dalam satu tempat.
 - ✓ **Penghindaran konflik nama** → Kelas dengan nama yang sama dapat berada di paket berbeda.
 - ✓ **Keamanan dan kontrol akses** → Dengan modifier seperti public, protected, dan private.
 - ✓ **Kode lebih mudah digunakan kembali** → Kelas dalam package bisa digunakan dalam proyek lain.

Terima Kasih

- Materi Selanjutnya
 - Class Diagram